semantic arts

Data-Centric Transformation Made Possible

www.semanticarts.com

OWL Restrictions

Best Practices and Design Patterns

Semantic Arts Ontology Knowledge Exchange

October 31, 2023

1

Included

- Values
  - `allValuesFrom` (AVF)
  - `someValuesFrom` (SVF)
- Cardinality
  - `minCardinality / minQualifiedCardinality`
  - `maxCardinality / maxQualifiedCardinality`
  - `cardinality / qualifiedCardinality`
- Not included:
  - `hasValue`
  - `hasSelf`
  - `oneOf`

2

Baseline Assumptions

- https://semarts.atlassian.net/wiki/spaces/TRR/pages/2423914497/Ontology+Engineering+Best+Practices
- SA-built ontologies are OWL 2 DL-compliant.
- Represent any OWL 2 DL-compatible semantics that you care about in OWL.
- Important semantic information that cannot be expressed in OWL 2 DL should be documented in an annotation.
- SHACL is not used in the ontology because:
  - It is not compatible with OWL 2 DL.
  - SHACL should be used to express data constraints rather than semantics.

3

min(Qualified)Cardinality

4

minCardinality 0

```
:Building
    a owl:Class ;
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
            :Place
            [
                a owl:Restriction ;
                owl:onProperty :hasWindow ;
                owl:minCardinality 0 ;
            ]
        ) ;
    ] ;
.
```

Has no logical implications.

Used as a hint that the class is often or typically associated with the property.

`gist:domainIncudes` is designed to express such hints.

5

Best Practice 1: minCardinality 0

- → 1a. Do not use `minCardinality 0` restrictions.
- → 1b. Use `gist:domainIncludes` to express class/property association hints.
- → 1c. Corollary: Also use `gist:rangeIncludes` where applicable.

6

SVF and minCardinality 1

```
gist:Collection
    a owl:Class ;
    rdfs:subClassOf [
        a owl:Restriction ;
        owl:onProperty gist:hasMember ;
        owl:minCardinality 1;
    ] ;
.
```

```
gist:Collection
    a owl:Class ;
    rdfs:subClassOf [
        a owl:Restriction ;
        owl:onProperty gist:hasMember ;
        owl:someValuesFrom owl:Thing ;
    ] ;
.
```


7

SVF and minQualifiedCardinality 1

```
gist:ControlledVocabulary
    a owl:Class ;
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
            gist:Collection
            [
                a owl:Restriction ;
                owl:onProperty gist:hasMember ;
                owl:onClass gist:Category ;
                owl:minQualifiedCardinality 1 ;
            ]
        ) ;
    ] ;
.
```

```
gist:ControlledVocabulary
    a owl:Class ;
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
            gist:Collection
            [
                a owl:Restriction ;
                owl:onProperty gist:hasMember ;
                owl:someValuesFrom gist:Category ;
            ]
        ) ;
    ] ;
.
```


8

Computational Properties of Cardinality Restrictions

- Non-simple property = transitive property or property chain.
  - Side note: A transitive property is just a shortcut for a property chain.
- A cardinality restriction with a non-simple property takes you out of OWL 2 DL.
- Try it in robot (https://robot.obolibrary.org/)
  - `robot validate-profile –profile DL –input gist.ttl –output validation.txt`
- This combination is therefore not used in our ontologies.
- Express the semantics in some other way if possible.

9

min(Qualified)Cardinality 1

- `owl:someValuesFrom owl:Thing` is logically equivalent to `owl:minCardinality 1`.
- `owl:someValuesFrom <class>` is logically equivalent to `owl:minQualifiedCardinality 1`.
- If we use `minCardinality`, we must consider whether the property is non-simple and use a different expression if it is.

10

Best Practice 2: minCardinality 1

- → Use `owl:someValuesFrom` instead of `owl:min(Qualified)Cardinality 1`.
- → You don't have to consider whether the restricted property is simple or non-simple, so:
  - → It is easier to use.
  - → It provides consistent formalism.

11

minCardinality > 1

```
gist:Person
    a owl:Class ;
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
            gist:LivingThing
             [
                 a owl:Restriction ;
                 owl:onProperty :hasParent ;
                owl:onClass gist:Person ;
                owl:minQualifiedCardinality 1 ;
             ]
        ) ;
    ] ;
.
```

More precise than `owl:someValuesFrom`.

But not OWL 2 DL-compliant unless the restricted property is simple.

12

Best Practice 3: minCardinality > 1

- → Use `minCardinality` with values greater than 1 if the restricted property is simple.
- → Use `owl:someValuesFrom` if the restricted property is non-simple.
  - → There will be loss of semantic precision.
  - → But it's better than nothing.
  - → Use an annotation like `skos:scopeNote` to provide semantics for human consumption.

13

Other Restrictions

14

Included

- owl:allValuesFrom
- owl:cardinality and owl:qualifiedCardinality
- owl:maxCardinality and owl:maxQualifiedCardinality
- First we consider the use of these restrictions inside an owl:equivalentClass axiom.
- Then we consider their use inside an rdfs:subClassOf axiom.

15

Cardinality Examples

```
gist:Person
    a owl:Class ;
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
                gist:LivingThing
                [
                    a owl:Restriction ;
                    owl:onProperty gist:hasBiologicalParent ;
                    owl:cardinality 2 ;
                ]
            ) ;
    ] ;
.
```

```
gist:USWorker
    a owl:Class ;
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
                gist:Person
                [
                    a owl:Restriction ;
                    owl:onProperty :ssn;
                    owl:maxCardinality 1 ;
                ]
            ) ;
    ] ;
.
```


16

allValuesFrom Example

```
gist:Parent
    a owl:Class ;
    owl:equivalentClass [
        a owl:Class ;
          owl:intersectionOf (
              gist:Person
              [
                  a owl:Restriction ;
                  owl:onProperty gist:hasChild ;
                  owl:allValuesFrom gist:Person ;
              ]
          ) ;
    ] ;
.
```


17

Axioms vs Data Integrity Constraints

In many cases, cardinality restrictions with values other than 1 are used to express *data integrity constraints* rather than *semantics.*

For example, if some US worker has more than one SSN, we would say the Social Security Administration has made an error or the data is faulty, not that the person does not work in the US.

We would not say that the person is not a person.

Scientific advances, real or conceivable, may also make the restriction invalid (cloning or …).

18

Real Semantic Restrictions

The cases in which such restrictions are semantically valid are fewer than you might think:

A binary star contains 2 stars (exact cardinality).

Triplets are 3 infants born at one birth (exact cardinality).

An agreement has at least 2 members (minimum cardinality).

A temporal relation connects as least 2 objects (minimum cardinality).

19

OWA and Inference Into Class

Given OWA, the restriction inside the equivalence prevents inference into the class.

You can infer something into the *complement* of the class.

The same holds true for the other restrictions in this section.

20

Best Practice 4: Restriction Inside Equivalence

→ Do not use `allValuesFrom`, `max(Qualified)Cardinalty`, or exact cardinality inside an `owl:equivalentClass` assertion.

21

Combining Equivalence to SVF and SubclassOf AVF

```
:Parent
    a owl:Class ;
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
            gist:Person
            [
               a owl:Restriction ;
               owl:onProperty :hasChild;
               owl:someValuesFrom gist:Person ;
            ]
        ) ;
    ] ;
```

```
rdfs:subClassOf
  [
      a owl:Restriction ;
      owl:onProperty :hasChild;
      owl:allValuesFrom gist:Person ;
  ] ;

.
```

The equivalence *defines* what it means to be a `:Parent`.

The subclassing *elaborates* on the definition and can also be used to infer the child into the `gist:Person` class.

22

Design Pattern 1: Definition and Elaboration

→ Use equivalence to SVF to *define* - infer something into - the class, and subclassing with AVF to *elaborate* on the meaning of the class and infer something into the object class of the AVF.

23

SVF and Property Ranges

24

Restating the Property Range in the Restriction

RESTATE THE RANGE

```
gist:hasAddress rdfs:range gist:Address .

gist:Building
    a owl:Class ;
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
            :Place
            [
                a owl:Restriction ;
                owl:onProperty gist:hasAddress;
                owl:someValuesFrom gist:Address ;
            ]
        ) ;
    ] ;
.
```

DO NOT RESTATE THE RANGE

```
gist:hasAddress rdfs:range gist:Address .

gist:Building
    a owl:Class ;
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
            :Place
            [
                a owl:Restriction ;
                owl:onProperty gist:hasAddress;
                owl:someValuesFrom owl:Thing ;
            ]
        ) ;
    ] ;
.
```


25

Object Specialization

The previous case is one of taste.

However, a good design pattern is to assert a high-level class in the property range, and a specialization in the restriction.

Although you should select a high-level class, use the most specific class that is true.

For example, this range assertion is not very useful:

```
gist:Address rdfs:subClassOf gist:Content .
gist:hasAddress rdfs:range gist:Content .
```

Instead use:

```
gist:hasAddress rdfs:range gist:Content .
```

```
gist:hasAddress rdfs:range gist:Address .
gist:StreetAddress rdfs:subClassOf gist:Address .

gist:Building
    a owl:Class ;
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
            :Place
            [
                a owl:Restriction ;
                owl:onProperty gist:hasAddress;
                owl:someValuesFrom gist:StreetAddress ;
            ]
        ) ;
    ] ;
.
```


26

Design Pattern 2: Property Ranges and Restrictions

→ Use the most specific range that is true of the property without making it too narrow and thus not reusable.

→ Include a subclass in the restriction to specialize the class.

27